

Prompt:

8/17/2026, 9:42:43 PM

lanjut tahap B

Response:

8/17/2026, 9:42:43 PM

Tentu. **Tahap B adalah bagian yang sangat penting**, karena di sinilah `market state` yang kita bangun pada Tahap A mulai diubah menjadi **distribusi probabilistik return masa depan**. Dengan kata lain, kita berpindah dari:

Apa yang sedang terjadi di pasar?

menjadi:

Apa yang secara probabilistik mungkin terjadi berikutnya?

Dan saya ingin menjaga disiplin desain kita: **Tahap B belum melakukan optimasi trading**. Ia hanya menghasilkan informasi yang nanti dikonsumsi optimizer.

Tahap B — Conditional Forecast & Dynamic Risk Model

Secara keseluruhan kita ingin membangun:

$$S_t \rightarrow P(\mathbf{R}_{t+\Delta} | S_t)$$

kemudian mengambil:

$$\mu_t = E[\mathbf{R}_{t+\Delta} | S_t]$$

dan:

$$\Sigma_t = \text{Cov}(\mathbf{R}_{t+\Delta} | S_t)$$

serta:

$$B_t = \text{factor loading}$$



sehingga nantinya:

$$\Sigma_t = B_t \Sigma_{F,t} B_t^T + D_t.$$

Inilah objek yang akan masuk ke Tahap C/D.

B.1. Mengapa kita membutuhkan distribusi, bukan sekadar prediksi harga?

Misalkan sistem mengatakan:

$$E[r_{\text{SOL},t+1} \mid S_t] = 0.5\%.$$

Sekilas ini terlihat bagus.

Tetapi ada dua kemungkinan:

Kondisi A

$$r_{t+1} \sim N(0.005, 0.002^2)$$

Kondisi B

$$r_{t+1} \sim N(0.005, 0.02^2)$$

Expected return keduanya sama:

$$\mu = 0.5\%.$$

Tetapi risk-nya sangat berbeda.

Karena itu optimizer kita tidak boleh hanya mendapatkan:

$$\mu.$$

Ia harus mendapatkan:

$$(\mu, \Sigma)$$

atau bahkan distribusi lengkap:

$$P(\mathbf{R}_{t+\Delta} \mid S_t).$$

B.2. Definisi formal

Misalkan:

$$\mathbf{R}_{t+\Delta} = \begin{bmatrix} R_{1,t+\Delta} \\ R_{2,t+\Delta} \\ \vdots \\ R_{N,t+\Delta} \end{bmatrix}$$

adalah vector return masa depan.

Kita ingin memodelkan:

$$\mathbf{R}_{t+\Delta} \mid S_t.$$

Secara umum:

$$\mathbf{R}_{t+\Delta} \mid S_t \sim D(\boldsymbol{\mu}_t, \Sigma_t, \Theta_t)$$

dengan Θ_t berisi parameter distribusi lain jika diperlukan.

Untuk versi pertama, kita dapat menggunakan:

$$\mathbf{R}_{t+\Delta} \mid S_t \approx N(\boldsymbol{\mu}_t, \Sigma_t)$$

tetapi ini adalah **working approximation**, bukan klaim bahwa return finansial benar-benar Gaussian.

B.3. Mengapa conditional?

Perhatikan perbedaan:

Unconditional

$$E[R_i]$$

hanya menggunakan sejarah return.

Conditional

$$E[R_{i,t+\Delta} | S_t]$$

mempertimbangkan keadaan pasar saat ini.

Ini jauh lebih sesuai dengan sistem kita.

Misalnya historical average return SOL:

$$E[R_{\text{SOL}}] = 0.1\%.$$

Tetapi ketika:

$$S_t = s_{\text{risk-off}},$$

mungkin:

$$E[R_{\text{SOL}} | S_t] = -0.3\%.$$

Sedangkan ketika:

$$S_t = s_{\text{risk-on}},$$

mungkin:

$$E[R_{\text{SOL}} | S_t] = 0.4\%.$$

Jadi:

$$\mu_t \neq \text{historical mean tetap.}$$

B.4. Kita mulai dari model factor

Sekarang kita gunakan struktur yang sudah kita tetapkan.

Untuk aset i :

$$r_{i,t} = \alpha_i + \beta_i^T \mathbf{f}_t + \epsilon_{i,t}$$

dengan:

$$\mathbf{f}_t = \begin{bmatrix} f_{1,t} \\ \vdots \\ f_{K,t} \end{bmatrix}.$$

Dalam bentuk vector:

$$\mathbf{r}_t = \boldsymbol{\alpha} + B\mathbf{f}_t + \boldsymbol{\epsilon}_t$$

dengan:

$$B = \begin{bmatrix} \beta_{11} & \cdots & \beta_{1K} \\ \vdots & \ddots & \vdots \\ \beta_{N1} & \cdots & \beta_{NK} \end{bmatrix}.$$

B.5. Apa arti B ?

Misalnya:

$$B = \begin{pmatrix} 1.2 & -0.4 & 0.1 \\ 0.8 & -0.2 & 0.3 \\ 1.5 & -0.6 & 0.05 \end{pmatrix}$$

dengan factor:

$$\mathbf{f} = \begin{bmatrix} \text{Equity} \\ \text{USD} \\ \text{Gold} \end{bmatrix}.$$

Maka untuk aset pertama:

$$r_1 = \alpha_1 + 1.2f_{\text{Equity}} - 0.4f_{\text{USD}} + 0.1f_{\text{Gold}} + \epsilon_1.$$

Jadi beta bukan sekadar indikator.

Ia merupakan **pemetaan matematis dari factor shock ke asset return**.

B.6. Penurunan expected return

Mulai dari:

$$\mathbf{r}_{t+1} = \boldsymbol{\alpha} + B\mathbf{f}_{t+1} + \boldsymbol{\epsilon}_{t+1}.$$

Kita mengambil conditional expectation:

$$E[\mathbf{r}_{t+1} \mid S_t].$$

Karena expectation linear:

$$= E[\boldsymbol{\alpha} \mid S_t] + E[B\mathbf{f}_{t+1} \mid S_t] + E[\boldsymbol{\epsilon}_{t+1} \mid S_t].$$

Jika:

$$\boldsymbol{\alpha}$$

dan B dianggap parameter:

$$= \boldsymbol{\alpha} + BE[\mathbf{f}_{t+1} \mid S_t] + E[\boldsymbol{\epsilon}_{t+1} \mid S_t].$$

Kita asumsikan:

$$E[\boldsymbol{\epsilon}_{t+1} \mid S_t] = 0.$$

Maka:

$$\boldsymbol{\mu}_t = \boldsymbol{\alpha} + B\boldsymbol{\mu}_{F,t}$$

dengan:

$$\boldsymbol{\mu}_{F,t} = E[\mathbf{f}_{t+1} \mid S_t].$$

Ini adalah persamaan fundamental Tahap B.

B.7. Perhatikan sesuatu yang sangat penting

Expected return aset:

$$\mu_{i,t}$$

sekarang berasal dari:

asset-specific component + factor-driven component.

Secara eksplisit:

$$\mu_{i,t} = \alpha_i + \sum_{k=1}^K \beta_{ik} \mu_{F_k,t}.$$

Contohnya:

$$\mu_{\text{NVDA},t} = \alpha_{\text{NVDA}} + \beta_{\text{SPX}} \mu_{\text{SPX},t} + \beta_{\text{USD}} \mu_{\text{USD},t} + \beta_{\text{Gold}} \mu_{\text{Gold},t} + \dots.$$

B.8. Tetapi ada masalah

Bagaimana kita memperoleh:

$$\mu_{F_k,t} = E[f_{k,t+1} | S_t]?$$

Kita membutuhkan **factor forecasting model**.

Ini berarti Tahap B sebenarnya mempunyai dua tingkat:

$$S_t \rightarrow \text{factor forecast} \rightarrow \text{asset forecast}.$$

Bukan:

$$S_t \rightarrow \mu_i$$

secara black-box.

B.9. Factor forecast

Untuk setiap factor k , kita dapat membuat:

$$\hat{f}_{k,t+1} = g_k(S_t) + \eta_{k,t+1}.$$

Sehingga:

$$\mu_{F_k,t} = g_k(S_t).$$

Maka:

$$\mu_t = \alpha + Bg(S_t).$$

Ini membuat arsitektur model sangat jelas.

B.10. Tetapi jangan langsung memakai model machine learning yang rumit

Untuk baseline, saya sangat menyarankan model linear/dynamic terlebih dahulu.

Misalnya:

$$f_{k,t+1} = c_k + \theta_k^T S_t + \eta_{k,t+1}.$$

Maka:

$$\mu_{F_{k,t}} = c_k + \theta_k^T S_t.$$

Dengan demikian:

$$\mu_t = \alpha + B(c + \Theta S_t)$$

untuk representasi matrix yang sesuai.

B.11. Kenapa baseline linear penting?

Karena nanti kita ingin menjawab:

Apakah peningkatan performa benar-benar berasal dari model yang lebih pintar?

Kita bisa membandingkan:

Historical Mean

vs.

Linear Factor Model

vs.

Nonlinear ML

vs.

Regime-dependent Model.

Tanpa baseline, kita tidak tahu apakah kompleksitas kita berguna.

B.12. Estimasi beta

Sekarang kita perlu memperoleh:

$$B_t.$$

Untuk satu aset:

$$r_i = \alpha_i + \beta_i^T f + \epsilon_i.$$

Kita memiliki observasi:

$$t = 1, \dots, T.$$

Dengan matrix design:

$$X = \begin{bmatrix} 1 & f_{1,1} & \cdots & f_{K,1} \\ 1 & f_{1,2} & \cdots & f_{K,2} \\ \vdots & \vdots & & \vdots \\ 1 & f_{1,T} & \cdots & f_{K,T} \end{bmatrix}$$

dan:

$$\mathbf{y}_i = \begin{bmatrix} r_{i,1} \\ r_{i,2} \\ \vdots \\ r_{i,T} \end{bmatrix}.$$

Model:

$$\mathbf{y}_i = X\boldsymbol{\theta}_i + \boldsymbol{\epsilon}_i$$

dengan:

$$\theta_i = \begin{bmatrix} \alpha_i \\ \beta_{i1} \\ \vdots \\ \beta_{iK} \end{bmatrix}.$$

OLS memberikan:

$$\hat{\theta}_i = (X^T X)^{-1} X^T \mathbf{y}_i$$

jika $X^T X$ invertible.

B.13. Tetapi beta tidak boleh dianggap konstan selamanya

Untuk short-term trading:

$$\beta_{i,t}$$

lebih masuk akal daripada:

$$\beta_i.$$

Karena hubungan:

$$\text{NVDA} \leftrightarrow \text{NDX}$$

atau:

$$\text{SOL} \leftrightarrow \text{BTC}$$

dapat berubah.

Maka kita menggunakan rolling estimation:

$$\hat{B}_t = \text{OLS}(\text{window ending at } t)$$

atau nantinya metode yang lebih canggih seperti exponentially weighted estimation atau state-space/Kalman filtering.

Untuk versi pertama:

sudah cukup.

B.14. Dynamic covariance

Sekarang bagian yang paling penting untuk optimizer.

Mulai:

$$\mathbf{r}_t = \boldsymbol{\alpha} + B_t \mathbf{f}_t + \boldsymbol{\epsilon}_t.$$

Covariance:

$$\Sigma_t = \text{Cov}(\mathbf{r}_t \mid S_t).$$

Karena:

$$\mathbf{r}_t = B_t \mathbf{f}_t + \boldsymbol{\epsilon}_t$$

untuk bagian random:

$$\text{Cov}(B_t \mathbf{f}_t + \boldsymbol{\epsilon}_t).$$

Dengan sifat covariance:

$$\text{Cov}(AX) = A \text{Cov}(X) A^T$$

maka:

$$\text{Cov}(B_t \mathbf{f}_t) = B_t \Sigma_{F,t} B_t^T.$$

Jika factor dan residual uncorrelated:

$$\text{Cov}(\mathbf{f}_t, \boldsymbol{\epsilon}_t) = 0,$$

maka:

$$\Sigma_t = B_t \Sigma_{F,t} B_t^T + D_t$$

dengan:

$$D_t = \text{Cov}(\epsilon_t).$$

Ini adalah persamaan yang sangat penting untuk seluruh proyek.

B.15. Mengapa decomposition ini sangat berguna?

Kita dapat memisahkan:

$$\text{Total Risk} = \text{Systematic Risk} + \text{Idiosyncratic Risk}.$$

Systematic:

$$\Sigma_t^{sys} = B_t \Sigma_{F,t} B_t^T$$

Idiosyncratic:

$$\Sigma_t^{idio} = D_t.$$

Jadi:

$$\Sigma_t = \Sigma_t^{sys} + \Sigma_t^{idio}.$$

B.16. Contoh sederhana

Misalkan:

$$B = \begin{bmatrix} 1.2 \\ 0.8 \end{bmatrix}$$

dan factor covariance:

$$\Sigma_F = [0.0004].$$

Maka:

$$B\Sigma_F B^T = \begin{bmatrix} 1.2 \\ 0.8 \end{bmatrix} (0.0004) \begin{bmatrix} 1.2 & 0.8 \end{bmatrix}$$

sehingga:

$$= \begin{bmatrix} 0.000576 & 0.000384 \\ 0.000384 & 0.000256 \end{bmatrix}.$$

Terlihat bahwa **satu common factor saja sudah menghasilkan covariance antar-aset**.

Ini adalah salah satu alasan factor model sangat natural untuk sistem kita.

B.17. Estimasi factor covariance

Kita perlu:

$$\Sigma_{F,t}.$$

Versi sederhana:

$$\hat{\Sigma}_{F,t} = \text{Cov}(\mathbf{f}_{t-W+1:t})$$

menggunakan rolling window.

Tetapi karena kita memodelkan masa depan:

$$\Sigma_{F,t} = \text{Cov}(\mathbf{f}_{t+\Delta} \mid S_t)$$

secara ideal.

Ini berarti kita nantinya juga membutuhkan **conditional volatility model**.

B.18. Volatility clustering

Untuk satu factor:

$$f_t = \mu_t + \epsilon_t$$

dan:

$$\epsilon_t = \sigma_t z_t$$



dengan:

$z_t \sim$ standardized noise.

Kemudian:

$$\sigma_t^2$$

dapat dimodelkan secara dinamis.

Contoh klasik:

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

yaitu GARCH(1,1).

Namun sekali lagi, saya **tidak akan menjadikan GARCH sebagai keharusan versi pertama**.

Kita dapat mulai dengan EWMA.

B.19. EWMA volatility

Definisikan:

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_{t-1}^2$$

dengan:

$$0 < \lambda < 1.$$

Unroll:

$$\sigma_t^2 = \lambda^2 \sigma_{t-2}^2 + \lambda(1 - \lambda) r_{t-2}^2 + (1 - \lambda) r_{t-1}^2.$$

Teruskan:

$$\sigma_t^2 \approx (1 - \lambda) \sum_{j=1}^{\infty} \lambda^{j-1} r_{t-j}^2.$$

Jadi return terbaru mendapatkan bobot terbesar.

Ini sangat sesuai dengan short-term trading.

B.20. Dynamic correlation

Volatility saja belum cukup.

Kita membutuhkan:

$$\Sigma_t = D_t^{1/2} R_t D_t^{1/2}$$

dengan:

$$D_t = \text{diag}(\sigma_{1,t}^2, \dots, \sigma_{N,t}^2)$$

dan:

$$R_t = \text{correlation matrix.}$$

Ini adalah bentuk yang sangat berguna:

$$\Sigma_t = D_t^{1/2} R_t D_t^{1/2}.$$

B.21. Mengapa ini penting?

Misalnya:

$$\sigma_1 = 2\%, \quad \sigma_2 = 3\%.$$

dan:

$$\rho_{12} = 0.8.$$

Maka covariance:

$$\Sigma_{12} = \rho_{12} \sigma_1 \sigma_2.$$

Sehingga:

$$\Sigma_{12} = 0.8(0.02)(0.03) = 0.00048.$$

Risk portfolio kemudian bergantung bukan hanya pada individual volatility, tetapi juga hubungan:

$$\text{Risk} \leftrightarrow \text{correlation.}$$

Ini nantinya menghasilkan coupling:

$$J_{ij}$$

dalam QUBO/Ising kita.

B.22. Factor model vs direct covariance

Sekarang ada dua cara untuk mendapatkan covariance.

Cara 1

Direct:

$$\Sigma_t^{direct}.$$

Cara 2

Factor:

$$\Sigma_t^{factor} = B_t \Sigma_{F,t} B_t^T + D_t.$$

Saya menyarankan **keduanya kita hitung**.

Karena nanti kita dapat menguji:

$$\Sigma_t^{direct}$$

vs.

$$\Sigma_t^{factor}.$$

Factor model biasanya jauh lebih menarik ketika:

$$N \gg K.$$

B.23. Mengapa?

Misalnya:

$$N = 100$$

aset.

Covariance matrix mempunyai:

$$\frac{N(N+1)}{2}$$

parameter unik.

Untuk:

$$N = 100 :$$

$$5050$$

parameter.

Sedangkan jika:

$$K = 10$$

factors, kita hanya membutuhkan struktur factor:

$$B \in \mathbb{R}^{100 \times 10}$$

plus:

$$\Sigma_F \in \mathbb{R}^{10 \times 10}$$

dan residual variances.

Jauh lebih terstruktur.

B.24. Regime dependence

Sekarang kita tambahkan ide yang menurutku sangat penting dari desain awal kita:

$$z_t$$

sebagai market regime.

Misalnya:

$$z_t \in \{1, 2, 3\}.$$

Kita dapat memiliki:

$$\mu_t^{(k)}$$

dan:

$$\Sigma_t^{(k)}$$

untuk setiap regime k .

Jika:

$$p_{k,t} = P(z_t = k | S_t),$$

maka:

$$\mu_t = \sum_k p_{k,t} \mu_t^{(k)}$$

B.25. Tetapi covariance mixture tidak sesederhana weighted average

Ini penting secara matematis.

Jika:

$$R | z = k \sim (\mu_k, \Sigma_k)$$

dan:

$$P(z = k) = p_k,$$

maka:

$$E[R] = \sum_k p_k \mu_k.$$

Namun covariance:

$$\text{Cov}(R) = E[\text{Cov}(R | z)] + \text{Cov}(E[R | z]).$$

Sehingga:

$$\Sigma = \sum_k p_k \Sigma_k + \sum_k p_k (\mu_k - \mu)(\mu_k - \mu)^T$$

dengan:

$$\mu = \sum_k p_k \mu_k.$$

Term kedua sering terlupakan.

Ia merepresentasikan **uncertainty akibat ketidakpastian regime**.

B.26. Ini sangat menarik untuk trading

Misalnya:

$$P(\text{risk-on}) = 0.5$$

dan:

$$P(\text{risk-off}) = 0.5.$$

Meskipun masing-masing regime memiliki volatility kecil, perbedaan expected return antar-regime dapat membuat unconditional conditional-on-state variance menjadi besar.

Artinya:

regime uncertainty itself is risk.

Ini sangat cocok untuk risk-aware optimizer kita.

B.27. Bagaimana kita menentukan regime?

Untuk baseline, kita dapat menggunakan fitur:

$$S_t^{macro} = [r_{\text{Gold}}, r_{\text{USD}}, r_{\text{SPX}}, r_{\text{VIX}}, \sigma_{\text{market}}, \dots].$$

Kemudian model:

$$P(z_t = k \mid S_t).$$

Ada beberapa pendekatan:

Model 1 — threshold

Sederhana, tetapi kasar.

Model 2 — clustering

k -means

atau Gaussian Mixture Model.

Model 3 — Hidden Markov Model

HMM

lebih menarik karena regime memiliki persistence.

Untuk sistem kita, **HMM adalah kandidat yang sangat bagus**, tetapi saya akan menjadikannya versi B2, bukan baseline B0.

B.28. Dynamic factor model

Sekarang kita bisa menyusun keseluruhan model:

$$\mathbf{r}_{t+1} = \alpha_t + B_t \mathbf{f}_{t+1} + \epsilon_{t+1}$$

dengan:

$$E[\mathbf{f}_{t+1} \mid S_t] = \boldsymbol{\mu}_{F,t}$$

dan:

$$\text{Cov}(\mathbf{f}_{t+1} \mid S_t) = \Sigma_{F,t}.$$

Sehingga:

$$\boldsymbol{\mu}_t = \boldsymbol{\alpha}_t + B_t \boldsymbol{\mu}_{F,t}$$

dan:

$$\Sigma_t = B_t \Sigma_{F,t} B_t^T + D_t.$$

Ini adalah **output utama Tahap B**.

B.29. Tetapi kita belum selesai

Karena target kita adalah **short-term trading**.

Expected return:

$$\mu_t$$

harus dibandingkan dengan biaya transaksi.

Misalnya expected gross return:

$$\mu_{gross}.$$

Expected cost:

$$C_t.$$

Maka:

$$\mu_{net,t} = \mu_{gross,t} - C_t.$$

Lebih lengkap:

$$C_t = C_{spread} + C_{fee} + C_{slippage} + C_{impact}.$$

Ini sangat penting.

Jika:

$$\mu_{gross} = 0.08\%$$

tetapi:

$$C = 0.10\%,$$

maka:

$$\mu_{net} = -0.02\%.$$

Secara trading:

jangan trade

meskipun forecast gross return positif.

B.30. Kita juga perlu probabilitas arah

Dari distribusi:

$$R \sim N(\mu, \sigma^2)$$

kita dapat memperoleh:

$$P(R > 0) = \Phi\left(\frac{\mu}{\sigma}\right).$$

Misalnya:

$$\mu = 0.002$$

dan:

$$\sigma = 0.01.$$

Maka:

$$P(R > 0) = \Phi(0.2) \approx 0.579.$$

Jadi expected return positif, tetapi probabilitas naik hanya sekitar 58%.

Ini jauh lebih informatif daripada:

"model memprediksi harga naik."

B.31. Bahkan kita dapat menghitung probability of hitting TP/SL

Ini akan sangat berguna di Tahap C/D.

Misalnya entry:

$$P_0.$$

Stop:

$$P_{SL}$$

dan target:

$$P_{TP}.$$

Kita ingin:

$$P(\text{TP hit before SL} \mid S_t).$$

Untuk proses diffusion sederhana, kita dapat menggunakan model first-passage probability.

Jika log-price mengikuti:

$$dX_t = \mu dt + \sigma dW_t,$$

dengan:

$$X_0 = 0,$$

upper barrier:

$$u > 0$$

dan lower barrier:

$$-l < 0,$$

maka probabilitas mencapai upper barrier sebelum lower barrier dapat diturunkan sebagai:

$$P_{TP} = \frac{1 - e^{-2\mu l/\sigma^2}}{e^{2\mu u/\sigma^2} - e^{-2\mu l/\sigma^2}}$$

untuk $\mu \neq 0$, dengan formulasi yang ekuivalen tergantung definisi barrier.

Untuk:

$$\mu = 0,$$

hasilnya menyederhana menjadi:

$$P_{TP} = \frac{l}{u + l}.$$

Ini belum kita gunakan sekarang, tetapi nantinya sangat berguna ketika menentukan TP/SL secara matematis.

B.32. Jadi Tahap B mulai terhubung langsung dengan ide awalmu

Kamu sebelumnya mengusulkan:

setiap trade memiliki risk \$1.

Sekarang kita dapat memisahkan:

Forecast

$$\mu_t, \Sigma_t$$

Risk

$$SL$$

Reward

$$TP$$



Probability

$$P(TP \text{ before } SL)$$

sehingga expected value dapat ditulis:

$$EV = P_{TP} G - (1 - P_{TP})L - C$$

dengan:

$$L = \$1$$

sebagai loss budget.

Ini nantinya menjadi salah satu objective yang sangat natural untuk trading engine.

B.33. Kita sekarang memiliki hierarki matematika

Perhatikan:

$$S_t$$

↓

$$\text{Factor Forecast}$$

↓

$$\mu_{F,t}, \Sigma_{F,t}$$

↓

$$B_t$$

↓

$$\mu_t, \Sigma_t$$

↓

$$P(TP), P(SL), EV$$

↓

Tahap C/D:

Optimization

Ini sangat bersih secara matematis.

B.34. Spesifikasi Tahap B versi pertama

Saya sarankan kita membuat **B0** terlebih dahulu:

$$\mathbf{r}_{t+1} = \alpha + B_t \mathbf{f}_{t+1} + \epsilon_{t+1}$$

dengan:

Factor mean

$$\mu_{F,t} = \text{rolling conditional estimate}$$

Factor covariance

$$\Sigma_{F,t} = \text{rolling/EWMA covariance}$$

Factor loading

$$B_t = \text{rolling OLS}$$

Residual covariance

Versi pertama:

$$D_t = \text{diag}(\sigma_{\epsilon_{1,t}}^2, \dots, \sigma_{\epsilon_{N,t}}^2)$$

Output

$$\mu_t = \alpha + B_t \mu_{F,t}$$

dan:

$$\Sigma_t = B_t \Sigma_{F,t} B_t^T + D_t.$$

B.35. Kemudian B1

Setelah B0 berhasil:

$$\text{EWMA dynamic volatility}$$

sehingga:

$$\sigma_{i,t}$$

dan:

$$\Sigma_{F,t}$$

menjadi time-varying.

B.36. B2

Kemudian:

$$\text{regime model}$$

misalnya:

$$P(z_t = k \mid S_t).$$

Kemudian:

$$\mu_t = \sum_k p_{k,t} \mu_t^{(k)}$$

dan:

$$\Sigma_t = \sum_k p_{k,t} \Sigma_t^{(k)} + \sum_k p_{k,t} (\mu_t^{(k)} - \mu_t)(\mu_t^{(k)} - \mu_t)^T.$$

B.37. B3

Barulah kita dapat mencoba model nonlinear:

$$\mu_t = f_{\theta}(S_t)$$

misalnya:

- Gradient Boosting,
- Random Forest,
- Neural Network,
- temporal model,
- atau model lain.

Tetapi model nonlinear tersebut **harus dibandingkan terhadap B0/B1/B2**, bukan langsung dianggap lebih baik.

B.38. B4 — Distributional forecast

Jika ingin lebih jauh:

$$R_{t+\Delta} \mid S_t$$

tidak lagi dianggap Gaussian.

Kita bisa menggunakan:

Student- t

untuk heavy tails:

$$R \sim t_{\nu}(\mu, \sigma).$$

Ini masuk akal secara finansial karena return memiliki tail risk yang sering lebih besar daripada Gaussian.

Kemudian:

$$P(R < -L)$$



dapat dihitung secara lebih realistis.

B.39. Dan ini sangat relevan dengan risk \$1

Misalkan dua model menghasilkan:

$$\mu = 0.3\%.$$

Model Gaussian:

$$\sigma = 0.5\%.$$

Model Student- t :

$$\sigma = 0.5\%, \quad \nu = 4.$$

Keduanya memiliki mean dan variance yang sama, tetapi tail probability berbeda.

Untuk trading dengan SL:

$$L = \$1,$$

perbedaan tail behavior tersebut dapat menghasilkan perbedaan besar pada:

$$P(SL).$$

Jadi pada akhirnya:

distributional modeling → better risk estimation.

B.40. Hal yang paling saya sukai dari desain ini

Sekarang sistem kita tidak lagi terlihat seperti:

"Saya punya AI yang memprediksi harga."

Ia menjadi:

State → Conditional Distribution → Risk/Reward Distribution → Optimization → Execution.



Itu jauh lebih dekat dengan cara kita seharusnya memformulasikan **systematic trading problem**.

B.41. Hubungan dengan quantum optimization

Dan sekarang kita bisa melihat dengan jelas mengapa QUBO/Ising akan muncul secara natural.

Tahap B menghasilkan:

$$\mu_t$$

dan:

$$\Sigma_t.$$

Kemudian Tahap C/D akan membuat objective seperti:

$$\max_x [\mu_t^T x - \lambda x^T \Sigma_t x - C(x) - P_{\text{risk}}(x)].$$

Perhatikan:

$$\Sigma_t$$

muncul pada quadratic term.

Artinya:

$$\Sigma_{ij,t}$$

akan menjadi coupling antara decision i dan j .

Secara konseptual:

$$\Sigma_{ij,t} \rightarrow Q_{ij,t} \rightarrow J_{ij,t}$$

dan akhirnya:

$$J_{ij,t} Z_i Z_j$$

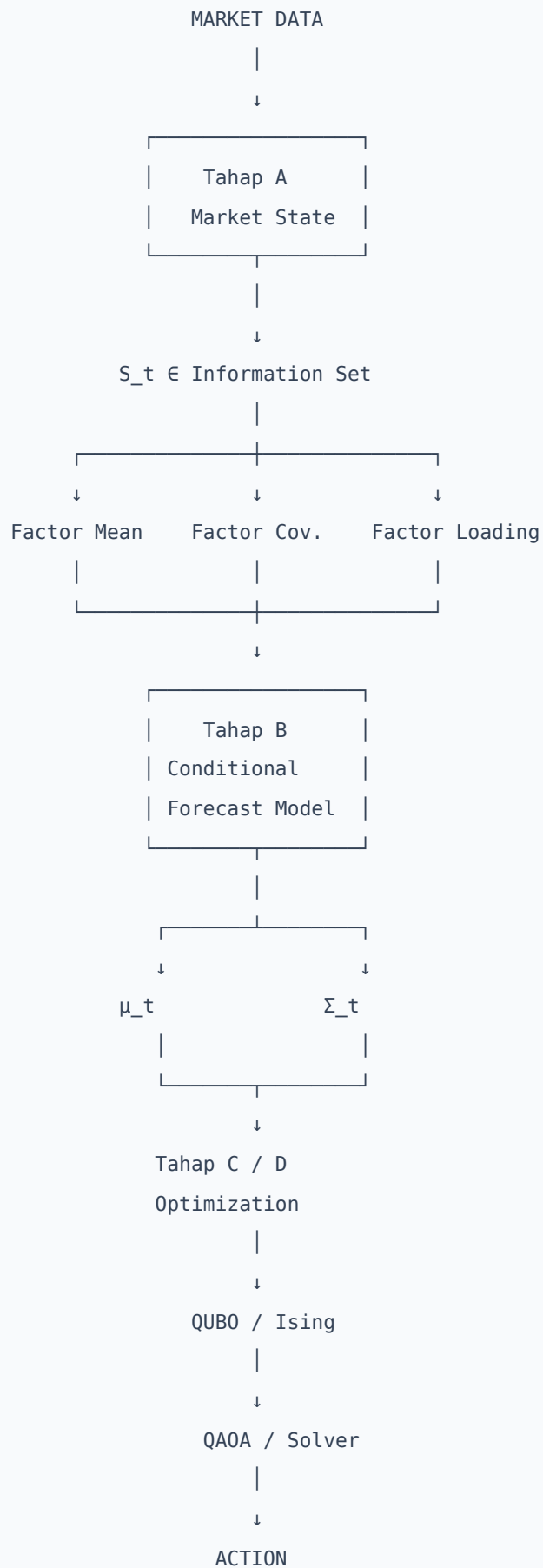
di Hamiltonian.

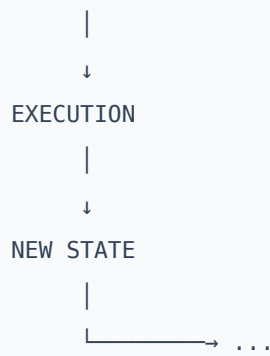
Jadi covariance yang kita bangun di Tahap B bukan sekadar statistik tambahan.

Ia pada akhirnya menjadi interaksi antar-qubit dalam Ising Hamiltonian kita.

Itulah hubungan yang sangat bagus antara finance dan quantum computing dalam arsitektur ini.

B.42. Pipeline yang sekarang sudah kita miliki





Posisi kita sekarang

Saya akan menetapkan **output resmi Tahap B** sebagai tiga objek:

$$B_t = \{\mu_t, \Sigma_t, B_t\}$$

dengan:

$$\mu_t = E[\mathbf{R}_{t+\Delta} | S_t]$$

$$\Sigma_t = \text{Cov}[\mathbf{R}_{t+\Delta} | S_t]$$

dan:

$$B_t = \text{exposure aset terhadap market factors.}$$

Untuk **versi implementasi pertama**, saya sangat menyarankan kita **jangan langsung memasukkan HMM, GARCH, neural network, mutual information, dan lain-lain**. Kita bangun dulu B0:

$$\text{Rolling Factor Regression + EWMA Volatility/Covariance}$$

dan pastikan ia dapat menghasilkan μ_t dan Σ_t yang masuk akal secara *out-of-sample*. Setelah baseline itu kokoh, baru kita tambahkan regime dan model nonlinear satu per satu.

Dan satu hal yang sangat penting: **Tahap B belum boleh "melihat masa depan" ketika melakukan rolling estimation**. Untuk setiap decision time t , seluruh parameter $B_t, \mu_{F,t}, \Sigma_{F,t}, D_t$ harus dibentuk hanya dari data F_t . Ini akan menjadi aturan keras ketika nanti kita masuk ke implementasi backtester.

Dengan fondasi ini, **Tahap C akan menjadi sangat menarik**, karena kita akan mengambil μ_t, Σ_t, B_t dan secara matematis menurunkan **masalah trading optimization yang sebenarnya**: mulai dari menentukan *universe*, long/short/flat decision, entry, position size, \$1 risk budget, transaction cost, factor exposure

constraint, hingga akhirnya membentuk **QUBO lengkap**. Dari sana barulah kita punya alasan matematis yang kuat untuk melakukan mapping:

Trading Problem $\rightarrow$ QUBO $\rightarrow$ Ising Hamiltonian $\rightarrow$ QAOA.
--